

# **A Tour of AI Skepticism Through the Lens of the Theory of Computation**

Workshop Slide Deck

---

Anonymous Author(s)

**Format:** Informal workshop / discussion

**No expectations** — but questions, discussion, and feedback are welcome!

**Pre-requisites:** Familiarity with P vs NP helps; otherwise, follow-up reading recommended

Introduction

The Church-Turing Thesis: What IS a Computer?

The Frame Problem

The Chomsky Hierarchy

NP-Completeness and Learnability

Discussion

# Introduction

---

# AI Skepticism: A Long Tradition

Skepticism about machines recreating human intelligence has deep roots:

- **Ada Lovelace** (1840s)
- **Noam Chomsky** — linguistics and cognition
- **Jerry Fodor** — cognitive science
- **John Searle** — Chinese Room argument
- **Roger Penrose** — physics and consciousness

Much of this skepticism can be framed in terms of the **Theory of Computation** – but often isn't.

This talk: fill that gap.

Advances in LLMs have prompted new skeptical arguments:

- **Bender & Koller (2020)** — “Climbing towards NLU”
- **van Rooij et al. (2024)** — “Reclaiming AI”

Today: reframe these arguments in *precise mathematical terms*

# Ada Lovelace's Objection

**Ada Lovelace** (1843) on Charles Babbage's Analytical Engine:

"The Analytical Engine has no pretensions to originate anything. It can do whatever we know how to order it to perform."



**John Searle (1980):**

## **The Argument**

- A person in a room follows a set of rules to manipulate Chinese symbols
- The person does not understand Chinese
- Therefore, syntactic manipulation does not imply semantic understanding

**Implication:** Symbol manipulation (computation) is not sufficient for understanding

## Computational perspective:

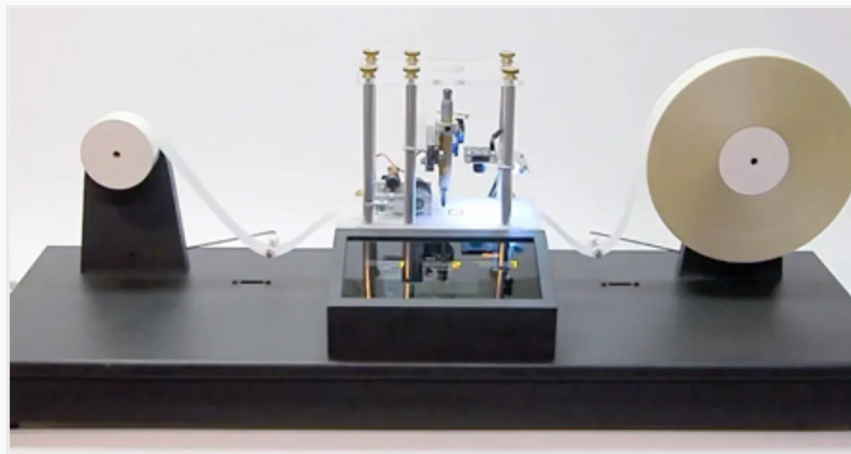
- A LARGE room
- An ENORMOUS amount of energy
- An UNBELIEVABLY COMPLEX set of rules
- Does your intuition about normal rooms really matter here?

# The Church-Turing Thesis: What IS a Computer?

---

- A implementation of what most people think of as a “computer”
- A tape of infinite length divided into cells
- A “head” that can read and write symbols on the tape
- The “head” can move left or right one cell at a time
- A finite table of rules that determine the head’s next action based on the current internal state and the symbol being read

# Turing Machines



<https://spectrum.ieee.org/032610-diy-turing-machine>

On the board: a simple Turing machine

# The Church-Turing Thesis

**Thesis:** Any function that is “effectively computable” can be computed by a Turing machine (equivalently, Python on a CPU)

## Implications for AI

- If cognition is computable  $\Rightarrow$  Turing machines can simulate it
- But: is cognition computable?

**Extended Church-Turing Thesis:** Any “reasonable” model of computation can be simulated by a Turing machine with at most polynomial slowdown. (e.g., quantum computers don’t violate this)

# What Turing Machines cannot do

Turing's Halting Problem and Goedel's Incompleteness Theorem, in VS Code

# Halting problem

On the board `https://www.youtube.com/watch?v=r24f3RmMjHs&t=1s`

# Goedel's Incompleteness Theorem

On the board `https://www.youtube.com/watch?v=91j\_JbIghM`

# Penrose's Objection

**Roger Penrose** (Physics Nobel Laureate):

Human mathematical insight transcends algorithmic computation

## **The Argument (roughly)**

- Gödel's incompleteness: no consistent formal system can prove all true statements
- Humans can “see” truths that no algorithm can prove
- Therefore: the mind is not algorithmic

Relies on quantum effects in microtubules (controversial, not widely accepted)

## Quick review:

- **P:** Problems solvable in polynomial time
- **NP:** Problems verifiable in polynomial time
- **NP-complete:** Hardest problems in NP

**The question:** Is  $P = NP$ ?

If  $P \neq NP$ , many problems (e.g., theorem proving) are intractable in the worst case.

**Reductions:** Transforming one problem into another

- If problem A reduces to problem B, solving B efficiently implies solving A efficiently
- NP-complete problems: all NP problems reduce to them
- Implication: if any NP-complete problem is in P, then  $P = NP$

## On the board: Halting Problem and Godel Incompleteness Theorem reductions

- Proofs on the board, following Aaronson notes style, using Python as model of computation.

# Goedel's Incompleteness Theorem

- Discussion
- J. Ellenberg's counterargument

## Reduction: Example

**Example:** 3-SAT reduces to CLIQUE

- 3-SAT: Given a boolean formula in CNF with 3 literals per clause, is there a satisfying assignment?
- CLIQUE: Given a graph and integer  $k$ , is there a clique of size  $k$ ?
- Reduction: Construct a graph from the 3-SAT instance such that a  $k$ -clique corresponds to a satisfying assignment

**Implication:** If we can solve CLIQUE efficiently, we can solve 3-SAT efficiently

## P, NP, and NP-Completeness: Examples

On the board: examples of problems in P, NP, and NP-complete

**Learning as search:** Many learning problems can be framed as search problems

- Finding a hypothesis consistent with training data
- Optimizing model parameters
- If these problems are NP-hard, efficient learning may be impossible
- Generally: arbitrary functions are **not** efficiently learnable (Valiant 1984)

**Open question:** Are there efficient algorithms for learning human-like intelligence?

# The Frame Problem

---

**Jerry Fodor:** How does a cognitive system determine what is *relevant*?

## The Problem

- Updating beliefs requires knowing what changes and what stays the same
- Potentially infinite number of things to check
- How do humans do this efficiently?

**Computational framing:** Relevance determination seems to require global computation  
**Computational framing:** Exponential time to compute: need to try all possible frames?

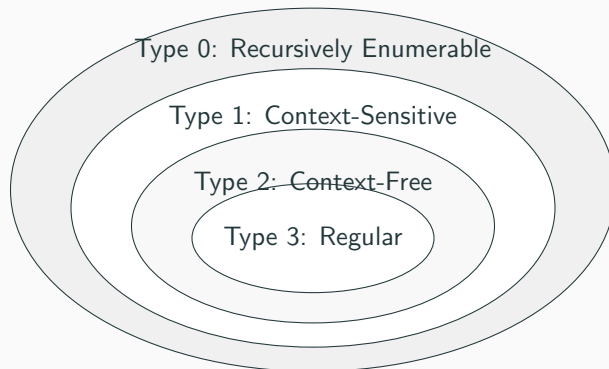
## Computational perspective:

- Naive approach: check all possible changes
- Exponential blowup in number of factors
- Humans seem to avoid this combinatorial explosion
- Is there a polynomial-time algorithm for relevance?
- Transformers determine the frame problem one token at a time (but maybe miss something (but so do humans?))

# The Chomsky Hierarchy

---

# The Chomsky Hierarchy



**Chomsky's claim:** Natural language requires at least context-sensitive grammars; statistical models miss the hierarchical structure.

On the board: examples of languages in each Chomsky hierarchy level. Arguments for why English is high on the level.

- Language is generative, rule-governed, hierarchically structured
- Statistical pattern matching is fundamentally different
- “Colorless green ideas sleep furiously” — grammatical but meaningless
- LLMs capture surface patterns, not underlying competence

**Computational question:** Can finite data reveal infinite generative capacity?

## **Gold's Theorem** (1967):

### **Result**

No algorithm can learn all recursively enumerable languages from positive examples alone

**Implication:** Learning human language may require more than just exposure to positive examples (e.g., innate constraints, negative evidence)

# NP-Completeness and Learnability

---

## P vs NP: Quick Review

- **P:** Problems solvable in polynomial time
- **NP:** Problems verifiable in polynomial time
- **NP-complete:** Hardest problems in NP

**The question:** Is  $P = NP$ ?

If  $P \neq NP$ , many optimization and learning problems are intractable in the worst case.

“Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data”

### Key Argument

- Meaning is grounded in communicative intent
- Form alone (text) cannot convey meaning
- LLMs learn form, not meaning
- “Octopus test”: can a system truly understand?

**Computational angle:** What can be learned from form alone?

- Bender and Koller (2020) slides: "[https://docs.google.com/presentation/d/17wP\\_0KEVSUqAtIg3aP4YWJAxJdZURxrOypgVPWwffbE/](https://docs.google.com/presentation/d/17wP_0KEVSUqAtIg3aP4YWJAxJdZURxrOypgVPWwffbE/)"
- Note: even if Physics argument is true, learning is not required to be tractable.

“Reclaiming AI as a Theoretical Tool for Cognitive Science”

**Claim**

Achieving human-like intelligence via learning from data is *intractable*

**Critique (Guerzhoy 2024):** The proof relies on unjustified assumptions about (input, output) distributions

*What assumptions are needed for intractability results?*

van Rooij et al. (2024), on the board.

# Barriers to Intractability Proofs

From Guerzhoy (2024), “Barriers to Complexity-Theoretic Proofs that Achieving AGI Using Machine Learning is Intractable”:

- Worst-case complexity  $\neq$  average-case complexity
- Distribution of real-world problems matters
- Current proof techniques may be insufficient

**Open question:** Can we prove (or disprove) fundamental limits on learning?

## Discussion

---

Theory of Computation provides a lens for AI skepticism:

- **Church-Turing Thesis:** What is computable?
- **Penrose:** Is the mind algorithmic?
- **Frame Problem:** Relevance and tractability
- **Chomsky Hierarchy:** Formal language theory and linguistic competence
- **NP-completeness:** Tractability of learning

These are *mathematical* questions with *philosophical* implications.

- What can be learned from finite data?
- Does scalability resolve intractability concerns?
- Is “understanding” computationally definable?
- What would constitute a proof of AI (im)possibility?

**Discussion welcome!**

## References

---

## Books:

- Aaronson, S. *Quantum Computing Since Democritus*

## Articles:

- Bender, E. & Koller, A. (2020). Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data. *ACL 2020*.  
<https://aclanthology.org/2020.acl-main.463/>
- van Rooij, I. et al. (2024). Reclaiming AI as a Theoretical Tool for Cognitive Science. *Computational Brain and Behavior*.  
<https://link.springer.com/article/10.1007/s42113-024-00217-5>

- Guerzhoy, M. (2024). Barriers to Complexity-Theoretic Proofs that Achieving AGI Using Machine Learning is Intractable.  
<https://arxiv.org/abs/2411.06498>
- Guerzhoy, M. (2026). Towards Formalizing Skepticism of Language Models: A Taxonomy in the Language of the Theory of Computation. PhilML Workshop at ICML 2026 (non-archival).

### Stanford Encyclopedia of Philosophy:

- The Frame Problem

<https://plato.stanford.edu/entries/frame-problem/>

### Additional background:

- Fodor, J. — Lectures 4–6 on the Frame Problem
- <https://slate.com/human-interest/2005/03/the-romantic-s-favorite-mathematician.html>

**Thank you!**

**Questions & Discussion**